

SUBMITTED BY: HARSHPREET KAUR (harshpreet.kaur@kenexai.com)

DATE OF SUBMISSION: 5 JUNE, 2026

› Question 1: Income Tax Calculator (Basic Operations)

↳ 2 cells hidden

› Question 2: Student Final Grade Calculator (Global & Local Variables)

↳ 2 cells hidden

✓ Question 3: String Slicing Operations

Description

Implement various string slicing operations to extract and manipulate string data.

Requirements

Complete the following tasks:

Extract Substrings:

- From "PythonProgramming", extract "Python"
- From "ABCDE12345XYZ", extract "12345"

Reverse Strings:

- Reverse "Hello, World!"
- Reverse "CS101_Introduction_to_Coding"

Extract User Information:

- Given "john_doe_1995", extract the first name ("john") and birth year ("1995")

Skip Characters:

- From "TheQuickBrownFoxJumpsOverTheLazyDog", extract every 2nd character

- From "LearningsFun123!", extract only the letters (remove numbers and special characters)

Input/Output Example Original String: PythonProgramming

Extracted: Python

Original String: ABCDE12345XYZ

Extracted: 12345

Original String: Hello, World!

Reversed: !dlroW ,olleH

Original String: john_doe_1995

First Name: john

Birth Year: 1995

Original String: TheQuickBrownFoxJumpsOverTheLazyDog

Every 2nd character: TeucBonoJmsvrhLzDg

Original String: LearningsFun123!

Letters only: LearningsFun

Constraints

- Use string slicing techniques, not built-in reverse functions
- Handle edge cases (empty strings, single characters)

```
# 1. Extract Substrings

str1 = "PythonProgramming"
print("Original String:", str1)
print("Extracted:", str1[:6])

str2 = "ABCDE12345XYZ"
print("\nOriginal String:", str2)
print("Extracted:", str2[5:10])

# 2. Reverse Strings

str3 = "Hello, World!"
print("\nOriginal String:", str3)
print("Reversed:", str3[::-1])

str4 = "CS101_Introduction_to_Coding"
print("\nOriginal String:", str4)
print("Reversed:", str4[::-1])
```

3. Extract User Information

```
user_info = "john_doe_1995"
first_underscore = user_info.index("_")
last_underscore = user_info.rindex("_")
first_name = user_info[:first_underscore]
birth_year = user_info[last_underscore + 1:]
print("\nOriginal String:", user_info)
print("First Name:", first_name)
print("Birth Year:", birth_year)
```

4. Skip Characters

```
str5 = "TheQuickBrownFoxJumpsOverTheLazyDog"
print("\nOriginal String:", str5)
print("Every 2nd Character:", str5[::2])
```

5. Extract Only Letters

```
str6 = "LearningIsFun123!"
letters_only = ""
for ch in str6:
    if ch.isalpha():
        letters_only += ch
print("\nOriginal String:", str6)
print("Letters Only:", letters_only)
```

Original String: PythonProgramming
Extracted: Python

Original String: ABCDE12345XYZ
Extracted: 12345

Original String: Hello, World!
Reversed: !dlroW ,olleH

Original String: CS101_Introduction_to_Coding
Reversed: gnidoC_ot_noitcudortnI_101SC

Original String: john_doe_1995
First Name: john
Birth Year: 1995

Original String: TheQuickBrownFoxJumpsOverTheLazyDog
Every 2nd Character: TeucBonoJmsvrhLzDg

Original String: LearningIsFun123!
Letters Only: LearningIsFun

Question 4: Collection Types - Inventory Management System

Description

Create an inventory management system that demonstrates operations on lists, dictionaries, sets, and tuples.

Requirements

1. List Operations Lists are ordered collections that can store multiple items. Use lists to maintain a simple list of product names.

Tasks:

- Create an empty list to store product names
- Add at least 3-4 products to the list
- Remove a product from the list (remove from a specific index)
- Update a product name at a specific index
- Display products:
- Show first 2 products
- Show last 2 products
- Show products from index 1 to 3

2. Dictionary Operations

Dictionaries store data as key-value pairs. Use dictionaries to store detailed product information where each product has multiple attributes.

Tasks:

- Create a dictionary where each key is a product name and value is another dictionary containing:
- name: product name (string)
- price: product price (number)
- quantity: available quantity (number)
- category: product category (string)
- Add at least 3 products to the inventory dictionary
- Update an existing product's quantity or price
- Delete a product from the inventory
- Search for a product by name
- Search for all products in a specific category

3. Set Operations

Sets store unique, unordered elements. Use sets to maintain a collection of unique product categories and perform set operations.

Tasks:

- Create a set to store unique product categories
- Add categories to the set
- Create two sets representing categories from two different stores (e.g., store1_categories and store2_categories)
- Perform the following set operations and display results:
- Find all categories from both stores
- Find common categories between stores
- Find categories only in first store
- Find categories in either store but not both

4. Tuple Usage

Tuples are immutable ordered collections. Use tuples to store fixed information like warehouse locations that shouldn't change.

Tasks:

- Create a dictionary that maps product names to their warehouse locations
- Each location should be a tuple containing exactly 3 elements:
- Section (e.g., "A-12")
- Shelf (e.g., "Shelf-3")
- Row (e.g., "Row-5")
- Display location information for at least 2 products
- Demonstrate tuple unpacking to extract individual location components (e.g., section, shelf, row = location)

Input/Output Example

```
=== Inventory Management System ===
```

```
--- List Operations ---
```

```
Initial Products: []
```

```
After adding products: ['Laptop', 'Mouse', 'Keyboard', 'Monitor']
```

```
After removing product at index 1: ['Laptop', 'Keyboard', 'Monitor']
```

```
First 2 products: ['Laptop', 'Keyboard']
```

```
Last 2 products: ['Keyboard', 'Monitor']
```

Products from index 1 to 3: ['Keyboard', 'Monitor']

--- Dictionary Operations ---

Adding Products:

Product added: Laptop (Price: Rs 50000,

Quantity: 10, Category: Electronics)

Product added: Mouse (Price: Rs 500,

Quantity: 50, Category: Electronics)

Product added: Keyboard (Price: Rs 1500,

Quantity: 30, Category: Accessories)

Updating Laptop quantity to 12...

Searching for 'Laptop':

Found: {'name': 'Laptop', 'price': 50000, 'quantity': 12, 'category': 'Electronics'}

Searching for products in 'Electronics'

category:

- Laptop
- Mouse

--- Set Operations ---

Store 1 Categories: {'Electronics', 'Accessories', 'Furniture'}

Store 2 Categories: {'Electronics', 'Books', 'Clothing'}

All Categories (Union): {'Electronics', 'Accessories', 'Furniture', 'Books', 'Clothing'}

Common Categories (Intersection): {'Electronics'}

Only in Store 1 (Difference): {'Accessories', 'Furniture'}

Unique to Each Store (Symmetric Difference): {'Accessories', 'Furniture', 'Books', 'Clothing'}

--- Tuple Operations ---

Laptop Warehouse Location: ('A-12', 'Shelf-3', 'Row-5')

Section: A-12, Shelf: Shelf-3, Row: Row-5

Mouse Warehouse Location: ('A-12', 'Shelf-3', 'Row-6')

Section: A-12, Shelf: Shelf-3, Row: Row-6

Constraints

- Validate all inputs
- Handle empty collections appropriately
- Use appropriate collection types for each scenario

```
print("=== Inventory Management System ===\n")
print("--- List Operations ---")

products = []
print("Initial Products:", products)
products.append("Laptop")
products.append("Mouse")
products.append("Keyboard")
products.append("Monitor")
print("After adding products:", products)

# Remove product at index 1
removed_product = products.pop(1)
print("After removing product at index 1:", products)

# Update product at index 1
products[1] = "Gaming Keyboard"
print("After updating product at index 1:", products)

# slicing
print("First 2 products:", products[:2])
print("Last 2 products:", products[-2:])
print("Products from index 1 to 3:", products[1:3])

print("\n--- Dictionary Operations ---")
inventory = {
    "Laptop": {
        "name": "Laptop",
        "price": 50000,
        "quantity": 10,
        "category": "Electronics"
    },
    "Mouse": {
        "name": "Mouse",
        "price": 500,
        "quantity": 50,
        "category": "Electronics"
    },
    "Keyboard": {
        "name": "Keyboard",
        "price": 1500,
        "quantity": 30,
        "category": "Accessories"
    }
}
```

```
}

print("Products Added:")
for product, details in inventory.items():
    print(details)

# Update Laptop quantity
inventory["Laptop"]["quantity"] = 12
print("\nUpdated Laptop quantity to 12")

# Delete Mouse
del inventory["Mouse"]
print("Deleted product: Mouse")

# Search by product name
search_product = "Laptop"

print(f"\nSearching for '{search_product}':")
if search_product in inventory:
    print("Found:", inventory[search_product])
else:
    print("Product not found.")

# Search by category
search_category = "Electronics"

print(f"\nProducts in '{search_category}' category:")
found = False

for product, details in inventory.items():
    if details["category"] == search_category:
        print("-", product)
        found = True

if not found:
    print("No products found in this category.")

print("\n--- Set Operations ---")

categories = set()
categories.add("Electronics")
categories.add("Accessories")
categories.add("Furniture")
print("Unique Categories:", categories)
store1_categories = {
    "Electronics",
    "Accessories",
    "Furniture"
}

store2_categories = {
```



```
    "Electronics",
    "Books",
    "Clothing"
}

print("\nStore 1 Categories:", store1_categories)
print("Store 2 Categories:", store2_categories)

print("\nAll Categories (Union):")
print(store1_categories | store2_categories)

print("\nCommon Categories (Intersection):")
print(store1_categories & store2_categories)

print("\nOnly in Store 1 (Difference):")
print(store1_categories - store2_categories)

print("\nUnique to Each Store (Symmetric Difference):")
print(store1_categories ^ store2_categories)

print("\n--- Tuple Operations ---")
warehouse_locations = {
    "Laptop": ("A-12", "Shelf-3", "Row-5"),
    "Keyboard": ("B-05", "Shelf-1", "Row-2"),
    "Monitor": ("C-10", "Shelf-4", "Row-7")
}

# Display location for Laptop
location = warehouse_locations["Laptop"]

print("Laptop Warehouse Location:", location)
section, shelf, row = location
print("Section:", section)
print("Shelf:", shelf)
print("Row:", row)

print()

# Display location for Keyboard
location = warehouse_locations["Keyboard"]

print("Keyboard Warehouse Location:", location)

section, shelf, row = location

print("Section:", section)
print("Shelf:", shelf)
print("Row:", row)

=== Inventory Management System ===
```

```
--- List Operations ---
Initial Products: []
After adding products: ['Laptop', 'Mouse', 'Keyboard', 'Monitor']
After removing product at index 1: ['Laptop', 'Keyboard', 'Monitor']
After updating product at index 1: ['Laptop', 'Gaming Keyboard', 'Monitor']
First 2 products: ['Laptop', 'Gaming Keyboard']
Last 2 products: ['Gaming Keyboard', 'Monitor']
Products from index 1 to 3: ['Gaming Keyboard', 'Monitor']

--- Dictionary Operations ---
Products Added:
{'name': 'Laptop', 'price': 50000, 'quantity': 10, 'category': 'Electronics'}
{'name': 'Mouse', 'price': 500, 'quantity': 50, 'category': 'Electronics'}
{'name': 'Keyboard', 'price': 1500, 'quantity': 30, 'category': 'Accessories'}

Updated Laptop quantity to 12
Deleted product: Mouse

Searching for 'Laptop':
Found: {'name': 'Laptop', 'price': 50000, 'quantity': 12, 'category': 'Electronics'}

Products in 'Electronics' category:
- Laptop

--- Set Operations ---
Unique Categories: {'Electronics', 'Accessories', 'Furniture'}

Store 1 Categories: {'Electronics', 'Accessories', 'Furniture'}
Store 2 Categories: {'Books', 'Electronics', 'Clothing'}

All Categories (Union):
{'Books', 'Electronics', 'Accessories', 'Furniture', 'Clothing'}

Common Categories (Intersection):
{'Electronics'}

Only in Store 1 (Difference):
{'Furniture', 'Accessories'}

Unique to Each Store (Symmetric Difference):
{'Books', 'Furniture', 'Accessories', 'Clothing'}

--- Tuple Operations ---
Laptop Warehouse Location: ('A-12', 'Shelf-3', 'Row-5')
Section: A-12
Shelf: Shelf-3
Row: Row-5

Keyboard Warehouse Location: ('B-05', 'Shelf-1', 'Row-2')
Section: B-05
Shelf: Shelf-1
Row: Row-2
```

Question 5 Grade Classification System (Conditions)

↳ 2 cells hidden

> Question 6 Password Validation System (Iterations)

↳ 2 cells hidden

> Question 7: Data Processing with Loops (ETL Simulation)

↳ 2 cells hidden

> Question 8: Pattern Matching - Data Format Processor

↳ 2 cells hidden

> Question 9: String Manipulation - Text Analyzer

↳ 2 cells hidden

> Question 10: Comprehensive Calculator with All Concepts

↳ 2 cells hidden

